

CS3002E Compiler Design

Monsoon 2025

Lecture #33

Code Optimization - Data-flow Analysis

Department of Computer Science & Engineering
NIT Calicut

Identify redundant expressions, if any

```
t1=i*4  
x=a[t1]  
t2=i*4  
y=b[t2]
```

Identify redundant expressions, if any

```
t1=i*4  
x=a[t1]  
t2=i+1  
i=t2  
t3=i*4
```

Available Expressions

An expression $x + y$ is *available* at a *program point*¹ p , if every path from the entry node to p evaluates $x + y$, and after the last such evaluation, prior to reaching p , there are no assignments to x or y .

¹for each instruction, there is a program point before the instruction and a program point after the instruction

Available Expressions

List the expressions that are *available* at each *program point*.

```
t1=i*4
```

```
x=t1
```

```
t2=i+1
```

```
i=t2
```

```
t3=i*4
```

Identifying redundancy using Available Expressions

To check if an expression occurring in a statement is redundant, the set of available expressions at the input point of that statement is required.

In the below code, $i*4$ is *available* at the input point of the last statement and hence the expression $i*4$ in the last statement is redundant.

```
t1=i*4  
t2=a[t1]  
x=t2  
t3=i*4
```

Identify redundant computations, if any

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

Indicate the expressions *available* at each program point

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

Any redundant computations?

Available Expressions

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

- $i*4$ is evaluated in both the branches of the conditional.
- $i*4$ is *available* at the input point of the statement labelled L3, and hence occurrence of $i*4$ in this statement is redundant.

```
    if(i<10) goto L1
    goto L2
L1:t1=i*4
    t2=a[t1]
    x=t2
    goto L3
L2:t3=i*4
    t4=b[t3]
    x=t4
L3:t5=i*4
    c[t5]=x
```

- Identifying the redundancy requires information regarding the expressions that are computed along the control flow paths reaching L3.
 - the information is obtained by doing a *data-flow analysis* of the program.

Data-flow Analysis

- Derives information regarding the flow of data along program execution paths.
- Analysis is done in a *Flow Graph* representation of the program².
- Example: *Available Expression Analysis* - computes the set of expressions available at every program point. The information is required for doing Global CSE.

²we concentrate on analysis done within a single procedure

Analysis for computing available expressions

Computing **gen** and **kill** for Basic Blocks:

- A block *generates* expression $x + y$ if it evaluates $x + y$ and does not subsequently define x or y .
- A block *kills* expression $x + y$ if it assigns (or may assign) x or y and does not subsequently recompute $x + y$.

Analysis for computing Available expressions

For each basic block B , let

- gen_B be the set of expressions generated by B .
- $kill_B$ be the set of expressions killed by B .
- $IN(B)$ be the set of expressions available at the input point of B .
- $OUT(B)$ be the set of expressions available at the output point of B .

Analysis for computing Available expressions: Initializations

$$OUT(ENTRY) = \phi$$

For each basic block $B \neq ENTRY$,

$OUT(B) = U$, where U is the universal set of expressions.

Note: Every flow graph is assumed to have two empty basic blocks
- an ENTRY node which represents the starting node of the graph
and an EXIT node to which all exits out of the graph go.

Analysis for computing Available expressions: Data-flow equations.

$$OUT(B) = gen_B \cup (IN(B) - kill_B)$$

$$IN(B) = \bigcap_{P \text{ a predecessor of } B} OUT[P]$$

Iterative Algorithm to compute Available expressions

Input: A flow graph with gen_B and $kill_B$ computed for each block B .

Output: $IN[B]$ and $OUT[B]$ for each block B .

$OUT(ENTRY) = \phi;$

for (each basic block $B \neq ENTRY$)

$OUT(B) = U;$

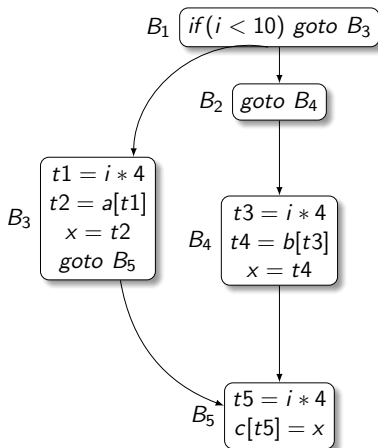
while (changes to any OUT occur)

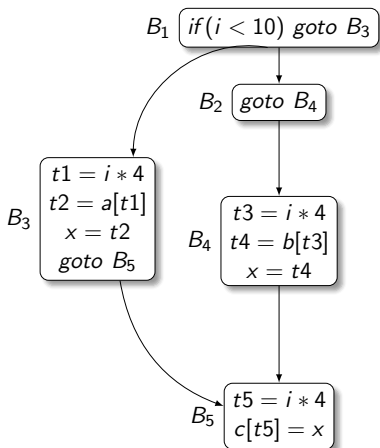
for (each basic block B other than $ENTRY$)

$IN(B) = \bigcap_{P \text{ a predecessor of } B} OUT[P];$

$OUT(B) = gen_B \cup (IN(B) - kill_B);$

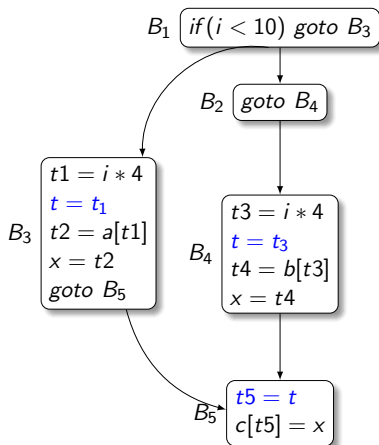
Exercise: Compute *gen*, *kill*, *IN* and *OUT* for each basic block





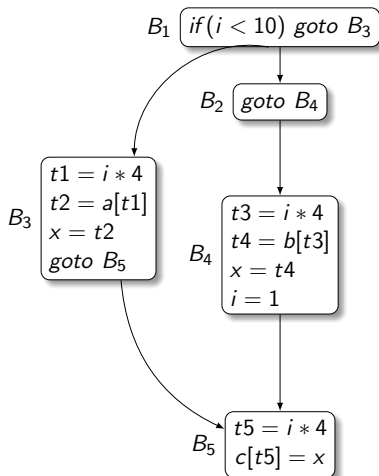
$IN[B_5] = \{i * 4\}$ and hence computation of $i * 4$ in B_5 is redundant.

Global Common Subexpression Elimination



The redundancy in B_5 is eliminated. Note the use of a new temporary t and the insertion of required copy statements.

Exercise: Compute IN and OUT for each basic block



Is $i * 4$ in B_5 redundant?

Topics for self-study

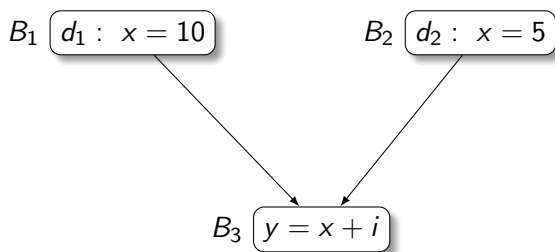
DAG representation of basic blocks and local optimizations - ALSU sections 8.5.1, 8.5.2 and 8.5.3.

Those interested to know more about data-flow analysis may read the rest of the material.

Reaching Definitions Analysis

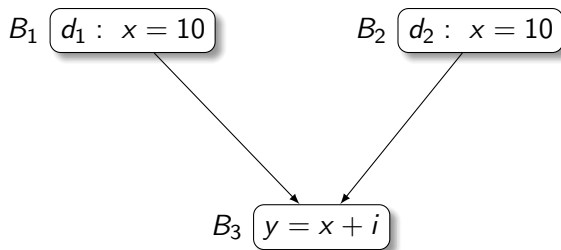
Computes for every program point, the set of definitions of variables that *may reach* that point.

Reaching Definitions



Reaching definitions at the entry point of B_3 : $\{d_1, d_2\}$

Reaching Definitions

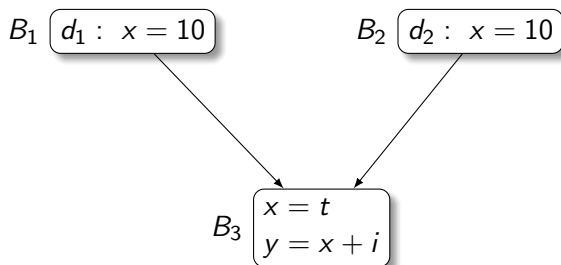


Reaching definitions at the entry point of B_3 : $\{d_1, d_2\}$.

the value of x used in B_3 is a constant.

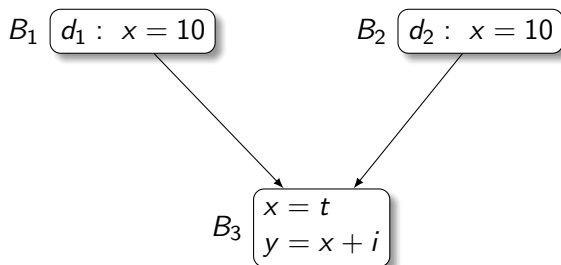
Is it possible to do *Constant Propagation*?

Reaching Definitions



Reaching definitions at the entry point of B_3 : $\{d_1, d_2\}$.
Is it possible to do *Constant Propagation*?

Reaching Definitions: *gen* and *kill*



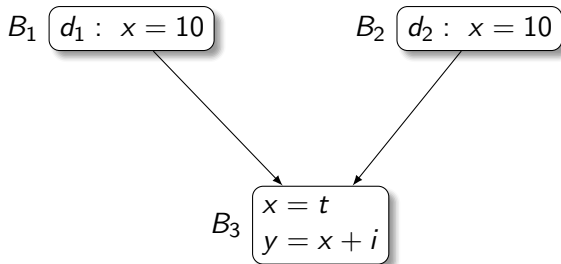
B_1 *generates* d_1 , B_2 *generates* d_2 .

$gen_{B_1} = \{d_1\}$

$gen_{B_2} = \{d_2\}$

$x = t$ *kills* definitions d_1 and d_2 .

Reaching Definitions: *IN* and *OUT*



$$OUT(B_1) = \{d_1\}$$

$$OUT(B_2) = \{d_2\}$$

$$IN(B_3) = OUT(B_1) \cup OUT(B_2) = \{d_1, d_2\}.$$

$x = t$ *kills* definitions d_1 and d_2 .

use of x in $y = x + i$ is not a constant.

Reaching Definitions: Data-flow equations

$$OUT(B) = gen_B \cup (IN(B) - kill_B)$$

$$IN(B) = \bigcup_{P \text{ a predecessor of } B} OUT[P]$$

Live Variable Analysis

A variable x is *live* at a program point p , if the value of x at p could be used along some path in the flow graph starting at p . Otherwise, x is *dead* at p .

Liveness information is required for *dead-code elimination* and for *Register Allocation* during code generation.

References

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU Section 8.4, Chapter 9